

LLM API Key Setup Guide

This document provides a collection of LLM providers offering Free Tier inference. These are ideal for building MVPs.

Best Practices

Environment Variables: Store keys in a .env file and load them using python-dotenv. Never push keys to public repositories.

Rate Limits: Most free tiers allow ~15–30 requests per minute. Implement retry logic or fallbacks or combination of multiple providers if you hit a 429 error.

1. Google Gemini (Google AI Studio)

Best for: Massive context windows (up to 2M tokens), long documents, and multimodal inputs.

Setup Instructions

1. **Get API Key:** Visit [Google AI Studio](#).
2. **Sign In:** Use your standard Google account.
3. **Generate Key:** Click "Create API key in new project."
4. **Install Library:**

```
pip install google-generativeai
```

Implementation Code

```
import google.generativeai as genai

genai.configure(api_key="YOUR_GEMINI_API_KEY")

# Models: 'gemini-2.5-flash' or 'gemini-1.5-flash' (fast/free) or
# 'gemini-1.5-pro' (complex)
model = genai.GenerativeModel('gemini-1.5-flash')

response = model.generate_content("What are the 3 main laws of
thermodynamics?")
print(f"Gemini Response: {response.text}")
```

2. Groq Cloud

Best for: Sub-second response times and high-speed inference for Llama and Mixtral models.

Setup Instructions

1. **Get API Key:** Visit [Groq Console](#).
2. **Generate Key:** Click "Create API Key" and copy it immediately.
3. **Install Library:**

```
pip install groq
```

Implementation Code

```
from groq import Groq

client = Groq(api_key="YOUR_GROQ_API_KEY")

# Models: 'llama-3.3-70b-versatile' (powerful) or 'mixtral-8x7b-32768'
# (fast)
completion = client.chat.completions.create(
    model="llama-3.3-70b-versatile",
    messages=[{"role": "user", "content": "Explain binary search in 2
sentences."}]
)

print(f"Groq Response: {completion.choices[0].message.content}")
```

3. GitHub Models (Marketplace)

Best for: Prototyping with diverse models like GPT-4o, Phi-4, and Llama 3 via a single GitHub account.

Setup Instructions

1. **Get Token:** Visit [GitHub Personal Access Tokens \(Fine-grained\)](#).
2. **Permissions:** Set "Account Permissions" > "Models" to Read-only.

3. **Documentation:** Refer to [GitHub Models Docs](#).
4. **Install Library:**

```
pip install azure-ai-inference
```

Implementation Code

```
from azure.ai.inference import ChatCompletionsClient
from azure.core.credentials import AzureKeyCredential

token = "YOUR_GITHUB_PAT"
endpoint =
"[https://models.inference.ai.azure.com](https://models.inference.ai.azure.
com)"

client = ChatCompletionsClient(endpoint=endpoint,
credential=AzureKeyCredential(token))

# Models: 'gpt-4o-mini', 'phi-4', or 'meta-llama-3.3-70b-instruct'
response = client.complete(
    messages=[{"role": "user", "content": "Write a Python function to sort
a list."}],
    model="gpt-4o-mini"
)
print(f"GitHub Response: {response.choices[0].message.content}")
```

4. Mistral AI

Best for: High-performance European models and specialized code-generation (Codestral).

Setup Instructions

1. **Get API Key:** Visit [Mistral Console](#).
2. **Generate Key:** Go to "API Keys" and click "Create new key."
3. **Install Library:**

```
pip install mistralai
```

Implementation Code

```
from mistralai import Mistral

client = Mistral(api_key="YOUR_MISTRAL_API_KEY")

# Models: 'mistral-large-latest' or 'codestral-latest'
chat_response = client.chat.complete(
    model="mistral-large-latest",
    messages=[{"role": "user", "content": "What is the capital of France?"}]
)
print(f"Mistral Response: {chat_response.choices[0].message.content}")
```

5. Ollama (Local LLM Setup)

Best for: Privacy, offline development, and zero-latency local testing.

Setup Instructions

1. **Download & Install:** Visit [Ollama.com Download](https://ollama.com/download).
2. **Server Setup:** Once installed, Ollama runs a local server at `http://localhost:11434`.
3. **Fetch a Model:** Open your terminal and run:

```
ollama run llama3.2:1b
```

Ollama Documentation: Ollama GitHub / API Docs.

Install Library:

```
pip install ollama
```

Implementation Code

```
import ollama

# Models: 'llama3.2:1b' (lightweight) or 'mistral' (powerful)
response = ollama.chat(model='llama3.2:1b', messages=[
    {'role': 'user', 'content': 'Explain local LLMs in one sentence.'},
])

print(f"Ollama Response: {response['message']['content']}")
```

Happy Coding!